

A COMPREHENSIVE MEASURE OF WELL-BEING

Project Documentation

Submitted as part of

APSCHE – SmartBridge AI & Machine Learning Internship

Project Title:

A Comprehensive Measure of Well-Being

Developed By

Team ID: _____

Team Members:

- Hema sree
 - Sai Saravan Gadamsetti
 - Pentyala Vaishnavi
 - Shivansh Chouhan
-

Guided By

SmartBridge – APSCHE Internship Program

Submission Date

07 July 2026

ChatFlow AI: Flask-Based Conversational Chatbot with LLM Integration

Project Description:

ChatFlow AI harnesses Generative AI to provide an intelligent, context-aware conversational assistant, offering users a natural and responsive chat experience. The platform includes a Conversation Engine, which maintains multi-turn dialogue context per session, an Intent & Tone Detector, which classifies each incoming message to shape the assistant's response style, and a Response Formatter, which structures the AI's reply along with suggested quick-reply options for the user.

Utilizing Groq's API with the LLaMA 3.3-70B Versatile model, ChatFlow AI processes user messages to deliver fast, coherent, and context-aware replies. Built with Flask and deployed publicly via Ngrok, the platform ensures a seamless and user-friendly experience. With secure API key management via .env, server-side session-based conversation memory, and responsible input handling, ChatFlow AI empowers support teams, product teams, and individual developers to deploy a capable chatbot with confidence.

Scenarios

Scenario 1: A user visiting a SaaS product's website opens the chat widget and asks about pricing plans. ChatFlow AI retrieves the conversation context, detects a Sales/Informational intent, and responds with a concise, friendly explanation of the plans along with a quick-reply suggestion to "Talk to Sales."

Scenario 2: A returning user reports that their previous file upload failed. ChatFlow AI recognizes a Support intent, references earlier turns in the same session to avoid asking redundant questions, and walks the user through troubleshooting steps in a calm, professional tone.

Scenario 3: A user chats casually, asking the assistant to recommend productivity tips. ChatFlow AI detects a Casual/General intent and responds in a warm, conversational tone with a short numbered list of tips, all generated in seconds.

Technical Architecture:

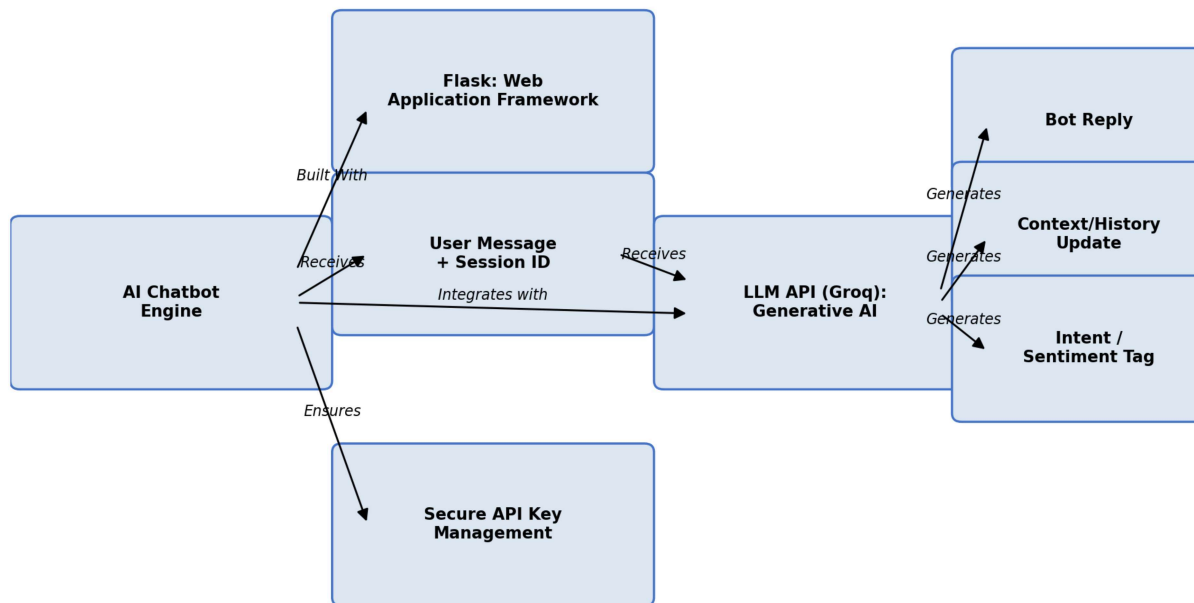


Figure 1: ChatFlow AI technical architecture

Description: ChatFlow AI utilizes a modular three-tier architecture to deliver responsive, AI-driven conversation. The frontend provides a responsive chat widget built with HTML, CSS, and JavaScript, while the Flask-based backend orchestrates session handling, input validation, and prompt construction. It interfaces with the Groq API using the LLaMA 3.3-70B model to generate context-aware replies, with secure deployment managed locally via Flask and exposed publicly through Ngrok.

(Note: If you are using a database in your project — for example, to persist chat history across sessions — definitely add an ER Diagram along with a description.)

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Groq API Familiarity:** Groq API Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Python Programming Proficiency:** Python Documentation
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Flask Installation Guide
- **Ngrok for Public Deployment:** Ngrok Documentation

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Generate a Groq API key and configure it securely in the project environment.
- **Activity 1.2:** Research and select the appropriate generative AI model from Groq for conversational content generation.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between the frontend, backend, session store, and AI integration.

- **Activity 1.4:** Set up the development environment, installing necessary libraries and dependencies for Flask and the Groq API.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Develop the core functionalities: Conversation Engine, Intent & Tone Detector, and Response Formatter.
- **Activity 2.2:** Implement the Flask backend to manage routing, session-based memory, and user input processing, ensuring smooth API interactions.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing routes for chat messaging and integrating AI responses.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the chat widget UI using HTML, CSS, and JavaScript, ensuring a responsive and intuitive layout.
- **Activity 4.2:** Create dynamic templates with Flask's render_template and JavaScript to display the running conversation based on user interactions.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and ensuring all dependencies are correctly installed.
- **Activity 5.2:** Deploy the application publicly using Ngrok to expose the local Flask server over a secure public URL.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate generative AI model from Groq for our conversational assistant. This involves researching the capabilities and performance of various models that Groq offers, ensuring that the chosen model aligns well with the application's objectives of generating fast, coherent, context-aware replies.

Activity 1.1: Generate a Groq API Key

Before the application can communicate with the Groq LLM, you need a valid Groq API key. Follow the steps below to create one and connect it securely to the project.

- **Step 1 — Create a Groq Account:** Visit console.groq.com and sign up for a free account using your Google account or email address.
- **Step 2 — Navigate to API Keys:** After logging in, click on your profile icon in the top-right corner and select "API Keys" from the dropdown menu, or go directly to the API Keys page.
- **Step 3 — Create a New API Key:** Click the "Create API Key" button. Give your key a descriptive name (e.g., chatflowai-key) so it is easy to identify later.
- **Step 4 — Copy and Store the Key:** Give a "Display Name" and (optionally) an expiration, then click "Submit." Copy the key immediately — you will not be able to view it again after closing the dialog.
- **Step 5 — Add the Key to Your Project:** Create a `.env` file in the root of your project directory and paste the key as shown below:

```
GROQ_API_KEY=your_copied_api_key_here
```

Step 6 — Verify the Key is Loaded: The `app.py` file loads this key automatically at startup using `python-dotenv`:

```
from dotenv import load_dotenv
load_dotenv()
client = Groq(api_key=os.environ.get("GROQ_API_KEY"))
```

Important — Never Commit Your API Key: Add `.env` to your `.gitignore` file to prevent accidentally pushing the key to a public repository:

```
.env
```

Activity 1.2: Research and Select the Appropriate Generative AI Model

- **Understand the Project Requirements:** Review the specific needs of the ChatFlow AI application, focusing on multi-turn context retention, response latency, and reply coherence.
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine available LLM models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to conversational tasks.
- **Select the Optimal Model:** Choose `llama-3.3-70b-versatile` for its balance of coherent dialogue quality and low-latency generation, set with a temperature of 0.7 and `max_tokens` of 1024 for natural, focused replies.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend (HTML, CSS, JavaScript), Flask backend, session store, and Groq API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application — sending a message, viewing streaming or immediate replies, and seeing quick-reply suggestions.
- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /chat, validates and sanitizes user input, retrieves session history, builds a structured prompt, and communicates with the Groq API.
- **Describe AI Integration Points:** Define how the backend constructs prompts using a SYSTEM_PERSONA description and the rolling conversation history, calls the Groq API, and returns the structured reply to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects> pip install -r requirements.txt
```

Install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

Install Groq SDK:

```
(myenv) D:\projects> pip install groq==0.11.0
```

Install python-dotenv:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

Set Up Application Structure: Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).

Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the three core features — Conversation Handling, Intent/Tone Detection, and Response Formatting — as a unified AI response driven by a single structured prompt.

- **Conversation Engine:** Maintains a rolling window of the last several turns per session so replies stay context-aware without unbounded prompt growth.
- **Intent & Tone Detector:** Classifies the incoming message (Support, Sales, Casual, etc.) so the model can calibrate tone and level of detail.
- **Response Formatter:** Returns a natural-language reply plus up to three short quick-reply suggestions relevant to the conversation.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in `app.py` for the home page and the chat messaging endpoint.

Process User Input:

- Create a chat form in `index.html` for users to submit their message.
- Use Flask's `request.get_json()` to retrieve JSON data submitted from the frontend via AJAX.
- Validate that message is non-empty and within a 1000-character limit before processing.

Integrate API Calls:

- Within the `/chat` route handler, call the `build_prompt()` function to construct a structured prompt combining the system persona and the session's conversation history.
- Submit the prompt to the Groq API using `client.chat.completions.create()` with the `llama-3.3-70b-versatile` model.
- Parse and validate the JSON response using `extract_json()`, which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the ChatFlow AI application. In this milestone, you'll set up each feature's functionality through Flask routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- / — Home page route that renders the main index.html chat interface

```
@app.route("/")
def index():
    return render_template("index.html")
```

- /chat — POST endpoint that accepts JSON input, invokes the AI, and returns a structured reply

```
@app.route("/chat", methods=["POST"])
def chat():
    data = request.get_json()
    message = data.get("message", "").strip()
    session_id = data.get("session_id", "default")

    if not message:
        return jsonify({"error": "Message is required."}), 400
    if len(message) > 1000:
        return jsonify({"error": "Input too long. Please keep it under 1000 characters."}), 400

    try:
        history = get_session_history(session_id)
        prompt = build_prompt(message, history)

        chat_completion = client.chat.completions.create(
            messages=[
                {
                    "role": "system",
                    "content": "You are a helpful conversational assistant. Always respond with "
                               "valid JSON only, no markdown formatting."
                },
                {
                    "role": "user",
                    "content": prompt
                }
            ],
            model="llama-3.3-70b-versatile",
            temperature=0.7,
            max_tokens=1024,
        )
        response_text = chat_completion.choices[0].message.content
        result = extract_json(response_text)

        if not all(k in result for k in ["reply", "quick_replies", "intent"]):
            raise ValueError("Invalid response structure from AI")

        update_session_history(session_id, message, result["reply"])

    return jsonify({
        "success": True,
```



```

        "reply": result["reply"],
        "quick_replies": result["quick_replies"],
        "intent": result["intent"],
        "session_id": session_id
    })
except ValueError as e:
    import traceback
    traceback.print_exc()
    return jsonify({"error": f"Failed to parse AI response: {str(e)}"}), 500
except Exception as e:
    import traceback
    traceback.print_exc()
    return jsonify({"error": f"Generation failed: {str(e)}"}), 500

```

Set Up Route Handlers for Each Feature:

- For the /chat route, implement logic that reads message and session_id from the incoming JSON body using request.get_json().
- Apply input validation: return a 400 error if message is empty or exceeds 1000 characters.
- Route AJAX requests from the frontend to the /chat endpoint, which processes data and returns a JSON response.

Define Prompt Engineering Helpers:

- SYSTEM_PERSONA: a short description that anchors the assistant's overall tone and role.

```

SYSTEM_PERSONA = (
    "friendly, knowledgeable, and concise. Adapts tone to the detected intent "
    "(Support, Sales, or Casual) while staying accurate and helpful."
)

```

- build_prompt(): assembles a structured prompt from the persona, recent history, and the new message, instructing the model to return strictly valid JSON.

```

def build_prompt(message: str, history: list) -> str:
    history_text = "\n".join(
        f"{turn['role']}: {turn['content']}" for turn in history[-6:]
    )
    return f"""You are a conversational assistant. Persona: {SYSTEM_PERSONA}

Conversation so far:
{history_text}

New user message: "{message}"

Respond ONLY with valid JSON in this exact format:
{{
  "reply": "your natural-language reply",
  "quick_replies": ["short suggestion 1", "short suggestion 2", "short suggestion 3"],
  "intent": "Support | Sales | Casual | General"
}}"""

```

Integrate the Groq AI Responses in Each Function:

- Call client.chat.completions.create() with a system message enforcing JSON-only responses and a user message containing the assembled prompt.

- `extract_json()`: a robust parser that first attempts `json.loads()`, then strips markdown code fences (````json` blocks), and finally uses a regex to locate a raw JSON object — ensuring reliable parsing across model response variations.

```
def extract_json(text: str) -> dict:
    """Extract JSON from LLM response, handling markdown code blocks."""
    try:
        return json.loads(text)
    except json.JSONDecodeError:
        pass

    match = re.search(r'```(?:json)?\s*([\s\S]*?)```', text)
    if match:
        try:
            return json.loads(match.group(1).strip())
        except json.JSONDecodeError:
            pass

    match = re.search(r'\{[\s\S]*\}', text)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass

    raise ValueError("Could not extract valid JSON from response")
```

- Validate that the parsed result contains all three required keys — `reply`, `quick_replies`, and `intent` — before returning it to the frontend.
- **Home route:** `/` → renders `index.html`. **`/chat`** (POST) → accepts JSON, returns `{success, reply, quick_replies, intent, session_id}`

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing chat interface for ChatFlow AI. This involves designing a clean, modern chat-widget layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display the running conversation, quick replies, and typing indicators based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the ChatFlow AI brand, a scrollable message list, and a message-input bar.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.
- Integrate Font Awesome icons for the send button and message status indicators, and Google Fonts (Inter) for clean typography.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a clean chat-bubble aesthetic, distinguishing user messages from bot messages by alignment and color.
- Use flexbox to keep the message list scrollable while the input bar stays pinned to the bottom of the viewport.
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Message Bubbles:** individual bubbles per turn, styled differently for user vs. assistant, each with a timestamp.
- **Quick-Reply Chips:** rendered as tappable pill-shaped buttons beneath the latest bot message that resend the chosen text as the next user message.
- **Typing Indicator:** an animated "..." bubble shown while waiting for the AI response.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Message Submission Asynchronously:

- Attach a submit event listener to the chat form that prevents default form submission and sends the message as a JSON POST request to /chat via the Fetch API.
- While waiting for a reply, disable the send button and show the animated typing indicator in place of a bot message.

Render Results Dynamically:

- Implement the renderMessage(role, text) and renderQuickReplies(list) functions in main.js, which dynamically create and inject DOM elements for each new turn.
- After rendering, automatically smooth-scroll the message list to the latest message using scrollIntoView().

Restore UI State After Generation:

- In the finally block of the async handler, re-enable the send button, remove the typing indicator, and refocus the input field regardless of success or failure.

Milestone 5: Deployment

In Milestone 5, the focus is on deploying the ChatFlow AI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Create and activate a virtual environment, then install all required dependencies from requirements.txt.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup:

```
GROQ_API_KEY=your_groq_api_key_here
```

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to "http://127.0.0.1:5050" to access the app.
- Interact with the chatbot across a full conversation — test Support, Sales, and Casual intents — to verify replies, quick replies, and context retention are all correctly generated, parsed, and rendered.

Activity 5.3: Public Deployment via Ngrok

Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from ngrok.com/download.
- Click on "Microsoft Store Installer" (Windows) or follow the platform-specific instructions.

Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The run_public.py script sets the authtoken automatically using:

```
D:\projects>ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Replace the TOKEN value in run_public.py with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running app.py directly, launch run_public.py to start both the Ngrok tunnel and the Flask server:

```
D:\projects>python run_public.py
```

- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:

```
=====
YOUR PUBLIC WEBSITE URL IS LIVE!
URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
=====
```

- Share this URL with anyone who can access your ChatFlow AI app directly from their browser without any installation.

Important Notes:

- **debug=False** is required in run_public.py to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart run_public.py (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart run_public.py to get a new URL when this happens.
- In-memory session history resets on server restart. For persistence across restarts, back the session store with a database or Redis instance (and add an ER Diagram if you do).

Exploring the Website's Web Pages:

Home / Chat Page:

Description: The single-page interface of ChatFlow AI serves as both the landing page and the chat tool. It features a clean, modern design, a header displaying the ChatFlow AI brand (chat-bubble icon + logo), and a tagline. The main section contains the scrollable message list, quick-reply chips, and the message-input bar all on one seamless page.

Input / Message Bar:

Description: A fixed bottom bar containing a text input (up to 1000 characters) and a send button. Pressing Enter or clicking Send submits the message. While the AI processes the request, an animated typing indicator appears in the message list in place of the next bot reply.

Conversation / Results Area:

Description: Revealed and updated dynamically as the conversation progresses, this area displays each turn as a chat bubble — user messages right-aligned, assistant replies left-aligned. Beneath the latest assistant reply, up to three quick-reply chips let the user continue the conversation with a single tap. The viewport automatically smooth-scrolls to the newest message after each turn.

Conclusion:

ChatFlow AI is a generative AI platform built with Flask and styled with a clean chat-widget UI that streamlines conversational support and engagement. It uses Groq's high-speed inference API with the LLaMA 3.3-70B Versatile model to generate context-aware replies and quick-reply suggestions across Support, Sales, and Casual intents. The project, which included model selection, development, and deployment via Ngrok, highlights how targeted AI and a structured framework can boost responsiveness for support teams and product teams, with potential for future scalability. Looking ahead, ChatFlow AI offers significant opportunities for expansion, such as integrating persistent conversation storage in a database, adding multi-language support, connecting to knowledge-base retrieval for more accurate answers, and implementing user authentication to save chat history across sessions. By continuously refining the underlying model prompts and enhancing the frontend accessibility, the platform is well-positioned to evolve from a specialized tool into a comprehensive conversational AI assistant for teams of all sizes.